

AFRILANG Stdlib

std/c/nlpjaccard

Français. Bibliothèque AFRILANG 'std/c/nlpjaccard' (niveau complex, catégorie complex). Elle expose 6 fonction(s) : similarity, tokenJaccard, distance, ngramJaccard, overlap, dice.

```
'import "std/c/nlpjaccard"' · 'use nlpjaccard'
```

```
- 'similarity(a text, b text) → number' - 'tokenJaccard(a text, b text) →  
number' - 'distance(a text, b text) → number' - 'ngramJaccard(a text, b  
text, n number) → number' - 'overlap(a text, b text) → number' - 'dice(a  
text, b text) → number'
```

English. AFRILANG library 'std/c/nlpjaccard' (tier complex, category complex). Exports 6 function(s): similarity, tokenJaccard, distance, ngramJaccard, overlap, dice.

```
'import "std/c/nlpjaccard"' · 'use nlpjaccard'
```

```
- 'similarity(a text, b text) → number' - 'tokenJaccard(a text, b text) →  
number' - 'distance(a text, b text) → number' - 'ngramJaccard(a text, b  
text, n number) → number' - 'overlap(a text, b text) → number' - 'dice(a  
text, b text) → number'
```

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	6	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/nlpjaccard' (niveau complexe, catégorie complexe). Elle expose 6 fonction(s) : similarity, tokenJaccard, distance, ngramJaccard, overlap, dice.

```
'import "std/c/nlpjaccard"' · 'use nlpjaccard'
```

```
- `similarity(a text, b text) → number`
- `tokenJaccard(a text, b text) → number`
- `distance(a text, b text) → number`
- `ngramJaccard(a text, b text, n number) → number`
- `overlap(a text, b text) → number`
- `dice(a text, b text) → number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/nlpjaccard"
use nlpjaccard
```

Niveau : complexe · Catégorie : Modules complexe (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- similarity(a text, b text) → number•
- tokenJaccard(a text, b text) → number•
- distance(a text, b text) → number•
- ngramJaccard(a text, b text, n number) → number•
- overlap(a text, b text) → number•
- dice(a text, b text) → number

4. Exemples d'utilisation

```
import "std/c/nlpjaccard"
use nlpjaccard
```

```
create result = similarity(/* args */)
say result
```

```
create result = tokenJaccard(/* args */)
say result
```

```
create result = distance(/* args */)
say result
```

```
create result = ngramJaccard(/* args */)
say result
```

```
create result = overlap(/* args */)
say result
```

```
create result = dice(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/nlpjaccard"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module nlpjaccard

```
function similarity(a text, b text) returns number  
end
```

```
function tokenJaccard(a text, b text) returns number  
end
```

```
function distance(a text, b text) returns number  
end
```

```
function ngramJaccard(a text, b text, n number) returns number  
end
```

```
function overlap(a text, b text) returns number  
end
```

```
function dice(a text, b text) returns number  
end  
end
```

English

1. Overview

AFRILANG library 'std/c/nlpjaccard' (tier complex, category complex). Exports 6 function(s): similarity, tokenJaccard, distance, ngramJaccard, overlap, dice.

```
'import "std/c/nlpjaccard"' · 'use nlpjaccard'
```

```
- `similarity(a text, b text) → number`  
- `tokenJaccard(a text, b text) → number`  
- `distance(a text, b text) → number`  
- `ngramJaccard(a text, b text, n number) → number`  
- `overlap(a text, b text) → number`  
- `dice(a text, b text) → number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/nlpjaccard"  
use nlpjaccard
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- similarity(a text, b text) → number•
tokenJaccard(a text, b text) → number•
distance(a text, b text) → number•
ngramJaccard(a text, b text, n number) → number•
overlap(a text, b text) → number•
dice(a text, b text) → number

4. Usage examples

```
import "std/c/nlpjaccard"  
use nlpjaccard
```

```
create result = similarity(/* args */)   
say result
```

```
create result = tokenJaccard(/* args */)   
say result
```

```
create result = distance(/* args */)   
say result
```

```
create result = ngramJaccard(/* args */)   
say result
```

```
create result = overlap(/* args */)   
say result
```

```
create result = dice(/* args */)   
say result
```

5. Best practices

- Prefer ``import "std/c/nlpjaccard"`` at the top of your file.
- Run ``afriLang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module nlpjaccard

```
function similarity(a text, b text) returns number
end
```

```
function tokenJaccard(a text, b text) returns number
end
```

```
function distance(a text, b text) returns number
end
```

```
function ngramJaccard(a text, b text, n number) returns number
end
```

```
function overlap(a text, b text) returns number
end
```

```
function dice(a text, b text) returns number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/nlpjaccard"`
- Vérification : `afriLang check`
- Documentation : <https://afriLang.dev/stdlib/std/c/nlpjaccard/>

Source (extrait)

```
module nlpjaccard

function similarity(a text, b text) returns number
end

function tokenJaccard(a text, b text) returns number
end

function distance(a text, b text) returns number
end

function ngramJaccard(a text, b text, n number) returns number
end

function overlap(a text, b text) returns number
```

```
end  
function dice(a text, b text) returns number  
end  
end
```